

# Präsent-Verkauf in Shopify ohne Theme-Codebase

Technische Anleitung: Sage-Stückliste → Shopify-App → App-Block auf der Produktdetailseite  
Stand: 16. September 2026 · Referenz: Vinsecco-Shop und geplante Übertragung auf weitere Händler

**Kern in einem Satz:** Der Kunde pflegt Präsente in Sage (Elternartikel + Stückliste). Eine von uns betriebene Shopify-App schreibt Parent, Kinder und Verknüpfungen in den Shop. Ein App-Block im Theme-Editor zeigt Inhaltsliste und Nährwert-Reiter. Niemand braucht Zugriff auf den Theme-Quellcode des Händlers.

## 1. Für wen dieses Dokument ist

| Rolle             | Was sie tun muss                                                                                                                                                                   | Was sie nicht tun muss                                                                   |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Kunde / Händler   | In Sage Sets und Stücklisten pflegen. App installieren. Im Theme-Editor den Präsent-Block auf die Produktseite ziehen. Produkttyp/Collection prüfen.                               | Kein HTML, kein Liquid, kein Mapping von Nährwerten, kein Git-Zugriff auf das Theme.     |
| Unsere Entwickler | Custom App bauen und hosten. Sage lesen. Shopify Admin API schreiben. Theme App Extension (Block) liefern. Metafield-Definitionen anlegen. Sync-Regeln und Qualitätstore umsetzen. | Theme des Kunden forken. Nährwerte auf das Set kopieren. Freitext-Inhaltslisten pflegen. |

## 2. Fachmodell (gilt für Vinsecco und den Kunden gleich)

Ein Präsent ist kein einzelnes „fettes“ Produkt, sondern drei Dinge:

- Parent** — das Set, das der Kunde kauft (SKU, Preis, Status, Marketing).
- Stückliste** — welche Kinder in welcher Menge im Karton liegen.

3. **Kinder** — eigene Shop-Artikel (Wein, Feinkost, Zubehör) mit eigenen Nährwerten, Herkunft, Allergenen.

Der Nährwert-Reiter listet die Kinder nacheinander. Zubehör ohne Spezifikation (Schale, Löffel) steht nur in der Inhaltsliste. Die Inhaltsliste darf nicht händisch vom Nährwert-Reiter abweichen — bei Vinsecco passiert das heute (Windswept in der Beschreibung, Chenin Blanc im Reiter). Genau das soll die App verhindern.

**Regel:** Parent verkauft. Kind erklärt. Nährwerte nie auf das Set kopieren.

### 3. Warum eine Shopify-App und kein Theme-Zugriff

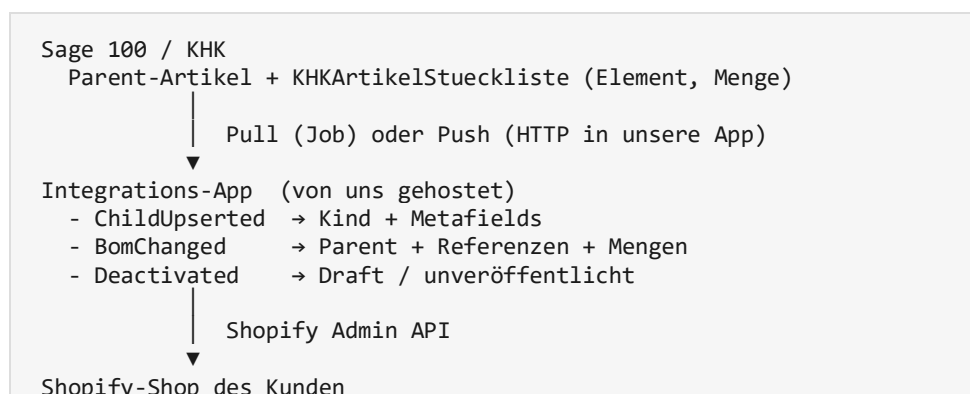
Beim Zielkunden haben wir keinen Zugriff auf deren Custom-Theme. Shopify-Webhooks allein reichen nicht: sie feuern, *nachdem* etwas im Shop passiert. Wir brauchen Sage → Shop.

In Shopify können wir ohne Theme-Git trotzdem drei Dinge nutzen:

| Mittel                          | Ort                                 | Zweck                                                            |
|---------------------------------|-------------------------------------|------------------------------------------------------------------|
| Custom data / Metafields        | Admin → Einstellungen → Custom data | Stückliste maschinenlesbar am Parent speichern                   |
| Admin API                       | Custom App                          | Produkte und Metafields anlegen/ändern                           |
| Theme App Extension (App Block) | Theme-Editor „Anpassen“             | Inhaltsliste und Reiter rendern, ohne Liquid des Shops zu ändern |

Voraussetzung: Theme **Online Store 2.0** (App Blocks). Bei Vintage-Themes geht nur ein Notbehelf über „Custom Liquid“ — das wollen wir vermeiden.

### 4. Zielarchitektur



```

Parent-Produkt (type Präsent)
Kinder-Produkte (Wein/Food)
Metafield custom.components = Liste Produktverweise
Metafield custom.component_qty = JSON (sku, qty, unit, label,
show_nutrition)
↓
Theme App Block auf der PDP
Inhaltsliste aus component_qty
Reiter 1: redaktioneller Text (Block-Setting)
Reiter 2: Loop über components, Wein- oder Food-Partial des Kindes

```

Die App spricht intern ein shop-neutrales Modell. Der Shopify-Adapter ist austauschbar. Sage und Stücklisten-Logik bleiben der Kern.

## 5. Technisches Vorgehen — Entwickler

### 5.1 App-Typ und Rechte

Für den einen Händler: **Custom App** (Dev Dashboard, Installation nur auf dessen Shop). Kein App Store nötig.

Minimale Admin-API-Scopes:

- read\_products / write\_products
- read\_inventory / write\_inventory (falls Set-Bestand mitgezogen wird)
- read\_metaobjects / write\_metaobjects nur falls später Metaobjects statt JSON
- Metafields: über Produkte; ggf. read\_metafield\_definitions / write\_metafield\_definitions

Theme App Extension separat im gleichen App-Projekt (Shopify CLI: App + Extension). Der Block wird nach Install im Theme-Editor sichtbar.

### 5.2 Custom Data einmalig anlegen

Die App soll das beim ersten Install selbst anlegen (nicht der Kunde per Hand, außer als Fallback):

| Metafield<br>(Namespace.key) | Owner               | Typ                    | Inhalt                                     |
|------------------------------|---------------------|------------------------|--------------------------------------------|
| custom.components            | Product<br>(Parent) | list.product_reference | Kind-Produkte in<br>Anzeigereihenfolge     |
| custom.component_qty         | Product<br>(Parent) | json                   | Menge, Einheit, Label,<br>Nährwert ja/nein |

|                        |                |                      |                                                                 |
|------------------------|----------------|----------------------|-----------------------------------------------------------------|
| bestehende Kind-Felder | Product (Kind) | wie Vinsecco-Mapping | jahrgang, alkohol_vol, brennwert_*, zutaten, herkunftsland, ... |
|------------------------|----------------|----------------------|-----------------------------------------------------------------|

Beispiel custom.component\_qty:

```
[
  {
    "sku": "605091",
    "qty": 1,
    "unit": "Flasche",
    "label": "Slow Chenin Blanc (False Bay)",
    "size": "0,75 l",
    "show_nutrition": true
  },
  {
    "sku": "820001",
    "qty": 1,
    "unit": "Stück",
    "label": "mintgrüne Keramikschale",
    "show_nutrition": false
  }
]
```

label und size kommen aus Sage-Stammdaten des Kindes (Bezeichnung, Flascheninhalt, Gewicht), nicht aus Freitext am Parent.

5.3 Zwei Sync-Schienen

| Schiene      | Trigger                        | Shopify-Aktion                                                                              |
|--------------|--------------------------------|---------------------------------------------------------------------------------------------|
| Artikel-Sync | Kind in Sage angelegt/geändert | Produkt upserten, Nährwert-/Herkunfts-Metafields schreiben                                  |
| Set-Sync     | Stückliste am Parent geändert  | Parent upserten, components + component_qty schreiben, Inhaltsliste ableiten, Status setzen |

Ein neuer Jahrgang am Wein braucht kein neues Set. Artikel-Sync reicht; alle Parents, die auf das Kind zeigen, zeigen automatisch neue Werte im App-Block.

## 5.4 Ereignismodell

| Ereignis           | Wann                                | Payload (kanonisch, ohne HTML)                    |
|--------------------|-------------------------------------|---------------------------------------------------|
| ChildUpserted      | Wein/Food/Zubehör gespeichert       | sku, titel, typ, metafield-map                    |
| BomChanged         | Stückliste gespeichert              | parent_sku, titel, active, components[{sku, qty}] |
| ArticleDeactivated | Artikel inaktiv / nicht mehr Online | sku, rolle parent   child                         |

Auslösung zuerst als **geplanter Pull** aus Sage (robust, wie Vinsecco-SQL).  
Push (Sage speichert → HTTP POST an die App) später als Optimierung.  
Beides verarbeitet dieselben drei Ereignisse. Idempotenz über SKU: zweimal derselbe Hook = gleiches Shop-Ergebnis.

## 5.5 Verarbeitungsreihenfolge (verbindlich)

1. Kinder per SKU in Shopify suchen oder anlegen.
2. Kind-Metafields schreiben (Mapping analog `metafield_mapping.py`).
3. Pflichtfelder je Typ prüfen (Wein: Alkohol, Herkunft, Füllmenge, Sulfite, Inverkehrbringer; Food: Nährwerte, Zutaten wo gesetzlich nötig).
4. Parent anlegen/updaten: Titel, SKU, Preis falls aus Sage, `product_type` = Präsent, Tag Präsent.
5. `custom.components` und `custom.component_qty` schreiben.  
Reihenfolge = Stücklistenreihenfolge.
6. Parent nur auf **active** setzen, wenn jedes Kind existiert und jedes `show_nutrition: true`-Kind spezifikationsvollständig ist. Sonst Draft und Fehlerlog.

Fehlt ein Kind oder ist es Draft: Parent bleibt unveröffentlicht. **Kein stilles Ersetzen** durch ein ähnliches Produkt derselben Marke (Vitsecco-Fehlerbild).

## 5.6 Sage-Anbindung (technisch)

Quelle wie bei Vitsecco, Mandant und Online-Flag analog beim Kunden klären:

- Eltern: Artikel mit Stücklistentyp, Artikelgruppe Set (bei Vinsecco 80100 / 880 / 800, SKU oft 8...).
- BOM: `KHKArtikelStueckliste.Stueckliste = Parent-SKU`, `Element = Kind-SKU`, Menge.
- Kinder: dieselben Stammdaten-Queries wie Wein/Food (Nährwerte am Kind, nicht am Set).

Query-Vorlage existiert bereits: `SQL_CoreData_Sets_Complete / Create_CoreDataSets_Complete()`. Heute schreibt sie nichts nach Shopify. Die App ist der fehlende Consumer.

Erkennung „ist Set“: Stückliste vorhanden **und** Artikelgruppe/Typ Set — nicht nur SKU-Prefix 8, weil der nächste Kunde andere Nummernkreise hat.

### 5.7 Theme App Extension (was der Block tut)

Ein App Block, z. B. „Präsent: Inhalt und Kennzeichnung“, Einstellungen im Editor:

- Nur anzeigen wenn `product.type == Präsent` (oder Metafield components nicht leer).
- Block 1: Inhaltsliste aus `component_qty` („1 Flasche Label – size“).
- Reiter A: Text aus Block-Setting (Händler kann den Schenken-Text selbst pflegen, ohne Deploy).
- Reiter B: für jede Referenz mit `show_nutrition` die Kind-Metafields rendern (Wein-Layout vs. Food-Layout anhand `product_type` oder LSA-Typ).

Der Block liest Storefront-sichtbare Metafields. Deshalb müssen die Definitionen **Storefront access / Liquid visible** haben, sonst sieht der Block nichts.

Shopify Bundles optional nur für Lagerabbau der Komponenten. Die PDP-Darstellung bleibt der App-Block; Bundles ersetzen keine LMIV-Reiter.

### 5.8 Was die App bewusst nicht tut

- Kein Schreiben einer händischen HTML-Inhaltsliste als führende Quelle.
- Kein Denormalisieren der Nährwerte auf den Parent.
- Kein Theme-File des Händlers überschreiben.
- Kein Shopify-Webhook als Auslöser für Neuanlage (nur optional Shop→App für Löschen/Status, falls gewünscht).

### 5.9 Hosting, Logs, Betrieb

- App-Backend in unserer Infrastruktur, Netz zu Sage (ODBC wie `Vinsecco_Client`).
- Job z. B. alle 15–60 Minuten Pull; oder Webhook-Endpoint der App für Sage-seitig ausgelöste POSTs.

- Protokoll: parent\_sku, fehlende Kinder, übersprungene Pflichtfelder, API-429-Retries.
- Dry-Run-Modus analog create-products --dry-run.

## 6. Vorgehen — Kunde / Händler

---

### 6.1 Voraussetzungen im Shop

1. Shopify-Plan mit Custom Apps und Online Store 2.0 Theme.
2. Einzelartikel (Wein/Food), die in Sets landen sollen, müssen als Produkte existieren oder mit dem Artikel-Sync mitkommen.
3. Keine parallele Pflege derselben SKUs von Hand im Admin, sonst überschreibt der Sync.

### 6.2 Arbeit in Sage (führendes System)

1. Kind-Artikel anlegen und pflegen (Nährwerte, Allergene, Herkunft am Kind).
2. Parent-Artikel als Verkaufs-Set anlegen, aktiv, für Online vorgesehen.
3. Stückliste am Parent: jedes Element mit Menge. Reihenfolge = gewünschte Reihenfolge im Shop.
4. Zubehör ohne Nährwert trotzdem als Artikel in der Stückliste führen, nicht nur im Kartontext.
5. Wein ersetzen = Stückliste ändern, nicht einen Text im Shop editieren.

### 6.3 Einmalig in Shopify

1. Von uns erhaltene Custom App installieren und API-Berechtigungen bestätigen.
2. Online Store → Themes → Anpassen → Produktvorlage.
3. App-Block „Präsent: Inhalt und Kennzeichnung“ unter die Kurzbeschreibung bzw. in den Reiter-Bereich ziehen.
4. Block-Setting: Anzeige auf Produkttyp Präsent beschränken. Reiter-1-Text bei Bedarf überschreiben.
5. Speichern. Danach gilt der Block für alle Präsent-Produkte dieser Vorlage.

### 6.4 Laufender Betrieb

Neues Präsent: nur Sage. Innerhalb des Sync-Intervalls erscheint der Parent als Entwurf oder live (je nach Qualitätstor). Inhaltsliste und Nährwert-Reiter entstehen automatisch. Der Händler prüft im Admin Stichproben (Kinder verknüpft? Block sichtbar? Draft-Gründe im Sync-Log bei uns erfragen).

## 6.5 Was der Kunde nicht tun soll

- Inhaltsliste in der Shopify-Produktbeschreibung von Hand pflegen (Vinsecco-Ist, hier Soll-Bruch).
- Nährwerte ins Set kopieren.
- Kind-Produkte löschen, die noch in Stücklisten hängen.
- Theme-Liquid ändern, um denselben Block nachzubauen.

## 7. End-to-End-Beispiel

---

Kunde legt in Sage Parent 824006 an, Stückliste:

1. 1 × 605091 Wein
2. 1 × 734103 Sauce
3. 1 × 732165 Zwiebeln
4. 1 × 735300 Gewürz
5. 1 × 820xxx Schale

App:

1. Stellt sicher, dass die fünf Kinder im Shop existieren und Metafields haben.
2. Legt/aktualisiert Parent 824006, type Präsent.
3. Schreibt vier Produktverweise mit Nutrition + Schale ohne Nutrition.
4. Publiziert nur wenn 605091/734103/732165/735300 spezifikationsvollständig sind.

Storefront: Block zeigt fünf Zeilen Inhaltsliste und vier Nährwert-Blöcke. Beschreibungstext des Parents darf Marketing sein („Südafrika für Genießer“), muss aber nicht die Stückliste wiederholen.

## 8. Abgrenzung zu Vinsecco heute

---

|           | Vinsecco jetzt              | Ziel mit App (Kunde und später Vinsecco) |
|-----------|-----------------------------|------------------------------------------|
| Auslösung | Cron Bestand + manuelle CLI | App-Job / Sage-Ereignis                  |

|                  |                                    |                                  |
|------------------|------------------------------------|----------------------------------|
| Kinder-Nährwerte | CSV + metafield_mapping, vorhanden | gleiche Logik in der App         |
| Parent aus BOM   | SQL lesbar, kein Writer            | App ist der Writer               |
| Inhaltsliste     | händisches body_html               | generiert aus component_qty      |
| PDP-Reiter       | fest im Theme                      | App-Block, vom Händler platziert |
| Theme-Git        | Vinsecco-Theme (fremd)             | nicht nötig                      |

## 9. Einführungsplan

---

1. Mit dem Kunden klären: OS-2.0-Theme, Sage hat Stücklisten, Nährwerte am Kind, Nummernkreis/Artikelgruppe für Sets.
2. Metafield-Vertrag und Pflichtfelder je Kind-Typ festziehen.
3. App: Sage-Pull + ChildUpsert + BomChanged, Dry-Run gegen einen Testshop.
4. App Block bauen, im Testshop auf Produktvorlage legen.
5. Ein Pilot-Set (3–5 Kinder, darunter Zubehör ohne Nährwert).
6. Qualitätstor prüfen (fehlendes Kind → Parent bleibt Draft).
7. Kunde installiert App, platziert Block, gibt Sage-Betrieb frei.
8. Erst danach weitere Sets und optional Push statt Pull.

## 10. Checklisten

---

### Entwickler vor Go-Live

- Scopes, Metafield-Definitionen inkl. Storefront-Sichtbarkeit
- Idempotenz SKU, Reihenfolge Kinder vor Parent
- Kein Publish bei fehlendem Kind
- Wein- vs. Food-Partial im Block
- Logs und Dry-Run
- Rate Limits Shopify 429

### Kunde vor Go-Live

- App installiert, Block auf Produkt-Template, gespeichert
- Pilot-Set in Sage vollständig (Stückliste + Kinderstammdaten)

- Im Shop: Inhaltsliste = Stückliste, Reiter = dieselben Kinder
- Klar: Änderungen nur in Sage, nicht in der Produktbeschreibung

## 11. Kurz das technische Vorgehen

---

1. Sage bleibt Quelle. Stückliste ist der Vertrag.
2. Custom App liest Sage und schreibt Shopify über die Admin API.
3. Kinder zuerst inkl. Metafields, dann Parent mit Produktverweisen und Mengen-JSON.
4. Theme App Block liest genau diese Metafields und baut Liste + Nährwert-Reiter.
5. Händler installiert App und zieht den Block im Editor — ohne Custom-Codebase.

Dokument für interne Abstimmung und Kundenübergabe. Keine Implementierung, nur Soll-Architektur.